

* find out all the list of companies which are conducting DSA

* what is the syllabus of GATE?

* int choice;

double price;

String itemname;

String itemcategory;

int itemcode;

unsigned phonenumber;

String Address;

String name;

unsigned int AN;

int Quantity;

* string source;

String destination;

double price;

* find out what are the set of variables in Amazon in voice bill.

* Identify Gaussba, current bill, set of datatypes & variables

string name;

unsigned int end;

double amount;

String Address;

The list of companies which are conducting DSA

Big Tech

- Google
- Amazon
- Microsoft
- Apple
- Netflix

arrays, strings,
trees, graphs,
dynamic programming

product based companies

- Adobe
- Oracle
- Intel

Service-Based companies (Basic + Moderate DSA)

- Tata consultancy services
- Infosys
- Wipro
- HCL Tech
- Tech mahindra
- Capgemini
- Cognizant
- Accenture

unsigned int order ID;

short int order date;

short int invoice date;

int quantity;

double price;

double totalamount;

String customername;

String Address;

String itemname;

String itemcategory;

int itemcode;

Indian product companies:

- Flipkart
- Phonepay
- Paytm
- Swiggy
- Meesho
- Zomato

Finance/Quant companies:

- Goldman Sachs
- JP Morgan Chase
- Morgan Stanley
- Tower Research Capital
- Jane Street

Topics of DSA:-

- | | | |
|-------------------------------|----------------------------|------------------|
| * Arrays | * Graphs | * Sliding window |
| * Strings | * Heaps | * Two pointers |
| * Linked lists | * Hashing | |
| * Stacks & Queues | * Recursion & Backtracking | |
| * Trees & Binary Search Trees | * Dynamic programming | |
| | * Binary search | |

GATE CSE:

- (1) Engineering Mathematics - Discrete Mathematics, Linear Algebra, Calculus, Probability & Statistics
- (2) Digital Logic: Boolean algebra, Combinational/Sequential circuits, Number representations, Computer arithmetic
- (3) Computer Organization & Architecture
- (4) Programming & Data Structures
- (5) Algorithms - Searching, Sorting, Hashing, Asymptotic complexity
- (6) Theory of Computation
- (7) Compiler Design
- (8) Operating System
- (9) Data bases
- (10) Computer networks

1) Structure in C and C++

Definitions

A structure (struct) is a user-defined datatype that allows us to group different types of variables under one name.

→ Instead of creating separate variables, we can store all these together in a structure.

Syntax in C:

```
struct student  
{  
    int rollno;  
    char name[30];  
    float marks;  
    char grade;  
};
```

To create a variable:

```
struct student s1;
```

In C we write `struct student s1;`. & In C++ we simply write `student s1;` because struct automatically becomes a new data type.

Syntax in C++:

```
struct student  
{  
    int rollno;  
    char name[30];  
    float marks;  
    char grade;  
};
```

To create a variable:

```
student s1;
```

Structures in C are mainly used to group related data, whereas structures in C++ are more powerful because they support object-oriented features such as member functions, constructors, destructors, inheritance and access specifiers.



2) Compute the size of the structure manually. use `sizeof` function and display the size. Crosscheck if both are same. Justify the answer.

consider the structure

```
struct Student  
{  
    int rollno;  
    char name[30];  
    float marks;  
    char grade;  
};
```

Size of each datatype

int $\rightarrow$ 4 bytes

char $\rightarrow$ 1 byte

float $\rightarrow$ 4 bytes

Manual calculation:

int rollno = 4 bytes

char name[30] = 30 bytes

float marks = 4 bytes

char grade = 1 byte

Total =

$4 + 30 + 4 + 1$

= 39 bytes

So Manual size = 39 bytes

check using `sizeof()`:

```
#include <iostream>  
using namespace std;  
struct Student  
{  
    int rollno;  
    char name[30];  
    float marks;  
    char grade; };
```

```

int main()
{
    cout << "size of student structure = " << sizeof(student)
        << "bytes";
    return 0;
}

```

3

Output :

Size of student structure = 40 bytes

Why is it 40 bytes instead of 39?

Because of structure padding (memory alignment).

Computers access memory faster when data is stored ^{addresses} aligned to certain byte boundaries (eg: 4-byte alignment for int and float).

To achieve this, the compiler may add extra unused bytes called padding.

Without padding: 39 bytes

compiler adds 1 padding byte? $39 + 1 = 40$ bytes

$\text{sizeof(student)} = 40$ bytes

eg 2:

struct test

```

{
    char a;
    int b;
}

```

};

Manual sizes

char = 1 byte

int = 4 byte

8) Find out the applications of recursion

Recursion is a programming technique in which a function calls itself repeatedly until a stopping condition is reached.

Applications:

1. Tree Traversal

Recursion is widely used to traverse tree data structures because every node can have child nodes that are processed in the same way.

Eg: Inorder Traversal, preorder Traversal, postorder Traversal
• Binary Trees, Binary Search Trees (BST), Expression Trees.

2. Graph Traversal

Many graph algorithms use recursion

Eg: Depth first search (DFS), finding connected components, detecting cycles in graphs, etc.

3. Mathematical Calculations

Recursion is useful for solving mathematical problems where each solution depends on a smaller problem.

Eg: Factorial, Fibonacci Series, Greatest Common Divisor (gcd), power Calculation.

4. Divide and Conquer Algorithms

Many efficient algorithms use recursion to divide a problem into smaller subproblems.

Eg: Merge Sort, Quick Sort, Binary Search

5. Backtracking problems

Backtracking is a recursive technique used to find all possible solutions.

Eg:

- N-Queens problem
- Rat in a Maze
- Sudoku solver
- Knight's Tour.
- Maze Solving

6. Directory and File System Traversal

Operating systems use recursion to search files and folders because directories contain subdirectories.

Eg: Display all files inside a folder & its subfolder

7. parsing Expressions

Compilers use recursion to analyze programming language syntax

→ Eg: Expression evaluation, Syntax tree generation, compiler design.

8. Artificial Intelligence

Eg. Decision Trees, Game Trees, Minimax algorithm, Alpha-Beta pruning.

8) Is recursion still used in the present programming era?
If there, where?

Yes, recursion is still widely used in modern programming. Although loops are preferred for simple repetitive tasks, recursion remains an imp't technique for solving complex problems involving hierarchical data structures, divide and conquer algorithms and backtracking.

Areas where recursion is used today

1. Software Development

→ Tree traversal, Graph traversal, Searching algorithms, parsing nested data

2. Web development

Modern web applications frequently process nested data structures such as JSON and XML using recursion

Eg: Displaying nested comments on social media.

processing API responses.

3. Operating Systems

→ Searching folders, file system traversal, directory management.

4. Database Systems

→ Recursive SQL queries, hierarchical employee records, Organisation charts

5. Artificial Intelligence

→ Chess engines, recursion Trees, Game solving, Search algorithms.

6. Compiler Design

→ parsing source code, building syntax trees, Evaluating expressions

7. Data Structures

→ Binary Trees, AVL Trees, Red-Black Trees, Tree, Graphs

8. Algorithms

→ Merge sort, Quick sort, DFS, Binary search

9. Mobile and Game development

→ 'path finding, puzzle solving, Game AI, Terrain generation

10. Scientific computing

→ Mathematical modeling, fractal generation, Simulation prgrms.

Q) What are the languages that support recursive calls?

A recursive call occurs when a function or method calls itself directly or indirectly. Most modern programming languages support recursion.

Languages that support recursive calls

1. C

→ supports recursive functions
→ used for mathematical calculations, trees and searching algorithms. Eg: factorial, fibonacci

2. C++

→ Supports recursion along with object oriented programming
→ Commonly used for trees, graphs, sorting & backtracking

3. Java

→ used in data structures, algorithms and application development.

4. python

→ widely used in Artificial Intelligence, ML & Data Science.

5. JavaScript

→ used in web development for processing nested HTML elements & JSON data

6. C#

→ supports recursion used in desktop applications, web applications & game development.

7. Kotlin

9. Go (Golang)

11. Functional programming language

8. Swift

10. Rust

12. PHP

13. Ruby

Q1) Find out the history of "towers of Hanoi"

Q2) Today from the mornig 7:30 AM to 4:30 PM, list out various data structures in the surroundings.

Q) History of Towers of Hanoi

The Tower of Hanoi is a famous mathematical puzzle created by the French mathematician Édouard Lucas in 1883. He introduced this puzzle to teach logical thinking & problem-solving skills.

Lucas based the puzzle on a legend. According to the story, there was a temple in Hanoi, Vietnam, where monks were given the task of moving 64 golden disks from one tower to another. They had to follow these rules:

- Move only one disk at a time
- Only the top disk can be moved
- A larger disk cannot be placed on a smaller disk.

The legend says that when the monks finish moving all 64 disks, the work will come to an end. This is only a story created to make the puzzle more interesting.

In reality, the Tower of Hanoi is a puzzle used in mathematics and computer science. It is one of the best eg to explain the concept of recursion in DSA. The puzzle teaches how a large problem can be solved by dividing it into smaller and simple problems.

Today the Tower of Hanoi is widely used in schools & colleges to help students improve their logical thinking, understand recursion and learn algorithm design. It remains one of the most popular puzzles in computer science.

Fun facts The min no. of moves needed to solve the puzzle with n disks is $2^n - 1$. For eg 3 disks need 7 moves and 4 disks need 15 moves.

Q) Data Structures in my surroundings

- 1) clothes arranged on shelves $\rightarrow$ array
- 2) Books stacked $\rightarrow$ stack
- 3) hostel room key chain (connected items) $\rightarrow$ linked list
- 4) students standing for breakfast in mess $\rightarrow$ queue (FIFO)
- 5) plates stacked one above another $\rightarrow$ stack (LIFO)
- 6) students sitting in seats $\rightarrow$ array
in bus
- 7) Bus stops connected in order $\rightarrow$ linked list
- 8) different departments connected by roads $\rightarrow$ graph
- 9) College building floors $\rightarrow$ Tree structure
- 10) Students sitting in rows $\rightarrow$ 2D array
in class
- 11) Attendance list $\rightarrow$ array/list
- 12) Students waiting to order food $\rightarrow$ queue
in canteen
- 13) Food trays stacked $\rightarrow$ stack
- 14) Books arranged on shelves in library $\rightarrow$ array
- 15) library catalogue $\rightarrow$ Tree / Binary search Tree
- 16) products arranged on racks in stationary / General store $\rightarrow$ array
- 17) Customers waiting to pay in shops $\rightarrow$ queue
- 18) Roads connecting hstl, cdy and shops $\rightarrow$ graph
- 19) Contact list in Mobile phone $\rightarrow$ linked list.

to estimate duplicate value

Ans!

```
#include <iostream>
using namespace std;
class student {
public:
    int rollno;
    float marks;
    char grade;
    int main()
{
```

student s1;

cout << "Size of student object = " <<

sizeof(s1) << "bytes" << endl;

return 0;

}

Output:

Size of student = 12 bytes

But in manually we have 29 bytes

Q1) find the frequency counts for the following

1) for ($i=1; i \leq n; i++$) {
 $\text{print}(\text{"Hello"});$
 $i \geq 1 \rightarrow 1$
 $i < n \rightarrow n-1$
 $i++ \rightarrow n-1$
 $\text{Hello} \rightarrow 1$
 $\text{Total} = 2n+1$

2) $i=1$

$i = i+20;$
 $\text{printf}(\text{"%.2f"}, i);$
 $i \geq 1$
 $i = i+20$
 frequency count
 1
 1
 $\text{Total} = \frac{1}{3}$

3) $i=1;$

while ($i \leq n$)
{
 $\text{printf}(\text{"%.2f"}, i); i++;$
}
 frequency count
 1
 $n+1$
 n
 $\text{Total} = \frac{3n+2}{2}$

Q2) for all shell programs done in OS lab, find out the time complexity

1) $\# / \text{bin} / \text{bash}$

$\text{echo} \text{"Hello world"} \rightarrow 1$

2) $\# / \text{bin} / \text{bash}$

$\text{echo} \text{"Enter num"} \rightarrow 1$

$\text{read num1} \rightarrow 1$

$\text{echo} \text{"Enter num"} \rightarrow 1$

$\text{read num2} \rightarrow 1$

$\text{sum} \$((\text{num1} + \text{num2})) \rightarrow 1$

$\text{echo} \text{"The sum of } \$\text{num1} \text{ \& } \$\text{num2} \text{ is } \$\text{sum}"}$

$\rightarrow 1+1+1+1+1+1$

$\rightarrow 6$

* Java Generates hashcode for every object find out the what is the Hashcode in Java & the purpose of generating the Hashcode.

In Java every object gets a hashcode. The hashcode is an integer value generated by the `hashCode()` method. Inherited from the `Object` class.

```

eg: class Student {
    public class Main
    {
        public static void main (String[] args)
        {
            Student s1 = new Student();
            Student s2 = new Student();
            System.out.println(s1.hashCode());
            System.out.println(s2.hashCode());
        }
    }

```

output:

1510467688
1995265320

```

eg: class Student {
    int id;
    Student(int id) {
        this.id = id;
    }
    public int hashCode()
    { return id; }
}

```

Purpose of hash code:-

- 1) fast searching:- collections like `HashMap`, `HashSet` and `Hashtable` use hashcodes to store and retrieve objects quickly.
- 2) Object Identification:- By default, different objects usually have different hashcodes, helping Java distinguish them.
- 3) Improves performance:- Instead of comparing every object one by one

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85
V	W	X	Y	Z																
86	87	88	89	90																

Java first compares hash codes, making lookups much faster. The main purpose of generating a hashcode is to enable efficient storage searching, and retrieval of objects in hash-based collections such as `HashMap` & `HashSet`.

The default `hashCode()` implementation in `Object` is typically based on the object's identity in memory.

Q What are the various problems which we observe in community. Quick any five problems find out if they can be solved using data structures and algorithm.

1. Traffic congestion :

- Graphs represent roads and intersections .
- Dijkstra's algorithm finds the shortest and fastest route .

2. Garbage collection :

- Algorithms determine the most efficient path for garbage trucks
- This reduces fuel consumption and time.

3. Hospital Appointment Management :

- A queue manages patients in order
- A priority queue gives emergency patients higher priority

4. Library Book Management :

- Hash Tables and Binary Search Trees help search books quickly
- Makes issuing and returning books efficient.

5. Bus route and Navigation :

- Bus stops are represented as nodes in a graph
- BFS or Dijkstra's algorithm finds the best route b/w two stops.

Trace Quick sort as shown in the classroom for the following array.

ch 85 76 99 13 45 103 -5 26 31

2) 350 799 400 200 100 96 900 890 760

0 1 2 3 4 5 6 7 8
85 76 99 13 45 103 -5 26 31

Pivot = 85

i j

j

85 76 31 13 45 103 -5 26 99

i j i j i j i j

85 76 31 13 45 26 -5 103 99

i j i j i j i j

-5 76 31 13 45 26 **85** 103 99

i j i j i j i j

-5 76 31 13 45 26 **85** **99** **103**

i i i i j

26 13 31 45 76

26 31 13 45 76

i i j j j

13 26 31 45 76

26 13 31 45 76

i i i j j

(ii) 350 790 400 200 100 96 900 890 760

P=350

i i j j j j

350 96 400 200 100 790 900 890 760

i i j j j j j j

350 96 100 200 400 790 900 890 760

i i j j i j j j j j

200 96 100 350 400 790 900 890 760

200 96 100 350 400 790 900 890 760

i i j i j j j j j

100 96 200 350 400 790 900 890 760

i j i j i j j j

96 100 200 350 400 790 760 890 900

j j j j j j

760 790 890 900

96 100 200 350 400 760 790 890 900